Week 2 - AWS Elastic Beanstalk (deployment policies, .ebextensions, swap URLs, worker environments)

DOP-C02 Domain 1 (SDLC Automation) and Domain 2 (Configuration Management and IaC). Exam focus: choosing the right deployment policy, configuring environments from code with .ebextensions, blue/green with swap URLs, and worker environments with SQS.

Part 1 - Building blocks

Term	Meaning
Application	A container that holds environments, versions and saved configurations
Application version	A labelled copy of your code: a **source bundle** (zip or war) stored in S3
Environment	One running copy of one version, with its own resources (ASG, ELB, security groups and so on)
Environment tier	**Web server** (behind an ELB) or **Worker** (pulls messages from SQS)
Platform	The OS and runtime (Node.js, Python, Java, .NET, Docker, Go and others). It has a branch (for example Python 3.12 on AL2023) and a version
Saved configuration	A stored template of environment settings, used to clone or recreate an environment

What Beanstalk creates for you

- An Auto Scaling group, launch template, ELB (ALB, NLB or CLB), security groups, CloudWatch alarms, an S3 bucket for bundles and logs, and an optional RDS database.
- Under the hood it is a **CloudFormation stack** (`awseb-e-xxxx-stack`).
- Change resources through Beanstalk, not directly in the EC2, ASG or ELB consoles. Direct changes cause drift and can be overwritten.

Environment types

- **Load-balanced** (ELB plus ASG): used for production.
- **Single instance**: one EC2 instance with an Elastic IP and no ELB. Cheap, for dev.
  - It supports only **All at once** and **Immutable** deployments. Rolling and traffic splitting need a load balancer and multiple instances.
- **The load balancer type cannot be changed after the environment is created** (for example Classic to ALB). Create a new environment and **swap URLs**.
- A **shared ALB** can be used by several environments, with host and path rules.

Source bundle rules

- One zip or war file, at most **500 MB**.
- Files go at the root, not inside a top-level parent folder.
- It may include `.ebextensions/`, `.platform/`, `Procfile`, `Buildfile`, `env.yaml` and `cron.yaml`.

Application version lifecycle policy

- An account can hold at most **1,000 application versions** per Region. When you reach the limit, new deployments fail.
- Set a lifecycle policy **by count** (keep the last N) or **by age** (delete versions older than N days).
- Optionally delete the source bundle from S3 too, or keep it.
- Versions that an environment is currently using are never deleted.

Roles

- **Service role** (`aws-elasticbeanstalk-service-role`): lets Beanstalk call other AWS services for you. Needed for enhanced health and managed updates.
- **Instance profile** (`aws-elasticbeanstalk-ec2-role`): the permissions the EC2 instances get, for example to read S3 or send messages to SQS. Your app's own permissions go here.

EB CLI commands

Command	What it does
`eb init`	Set up the application in a folder
`eb create`	Create an environment
`eb deploy`	Deploy the current code
`eb swap`	Swap URLs between two environments
`eb setenv`	Set environment properties
`eb config`	Change or save the configuration
`eb health`	Show health
`eb logs`	Get logs
`eb clone`	Copy an environment
`eb terminate`	Delete an environment

Other operations

- **Rebuild environment:** deletes and recreates all resources.
- **Restart app server:** restarts the app without replacing instances.
- A terminated environment can be **restored within 6 weeks**.

Part 2 - Deployment policies (application version deployments)

Set in namespace `aws:elasticbeanstalk:command`:

- `DeploymentPolicy`: one of `AllAtOnce`, `Rolling`, `RollingWithAdditionalBatch`, `Immutable` or `TrafficSplitting`.
- `BatchSizeType`: `Percentage` or `Fixed`, plus `BatchSize`.
- `Timeout`: the command timeout, **default 600 seconds**. Raise it if long deployments fail with timeouts.
- `IgnoreHealthCheck`: deploy even if health checks fail. Not recommended.

Policy	How it works	Downtime	Capacity	Failure and rollback	Extra cost
All at once	New version goes to all instances at the same time	**Yes**	Drops to zero	Every instance broken. Redeploy the old version manually	None
Rolling	One batch at a time: out of the ELB, update, back in	No	**Reduced** by one batch	Mixed versions. Manual redeploy	None
Rolling with additional batch	Launches one **extra batch** first, then rolls through the rest	No	**Full**	Mixed versions. Manual redeploy	Small (one batch)
Immutable	**Temporary ASG**: one new instance, then the full set; moved into the original ASG; old instances terminated	No	**Double** briefly	**Fastest and safest**: only the temporary ASG is terminated	High
Traffic splitting	Full new set in a temporary ASG; the **ALB sends a percentage** for an **evaluation time**, then 100%	No	Double	**Automatic rollback** if health falls	High

Traffic splitting

- Namespace `aws:elasticbeanstalk:trafficsplitting`: `NewVersionPercent` (for example 10) and `EvaluationTime` in minutes (for example 15).
- It **requires an ALB**. It is the only Beanstalk policy that is a true canary.

How to read exam questions

The question says	Answer
Fastest, dev, downtime OK	All at once
No extra cost, reduced capacity OK	Rolling
Must keep full capacity, minimal extra cost	Rolling with additional batch
Failure must not affect current instances, quickest rollback	Immutable
Test on X% of live users first, roll back automatically	Traffic splitting
Separate environment, instant rollback, platform branch or LB type change	Blue/green (swap URLs)

- **Rolling and failure:** Beanstalk does **not** roll back a rolling deployment automatically. Deploy the previous version again.

Configuration changes: rolling updates are different from deployment policies

- **Deployment policies** apply when you deploy a **new application version**.
- **Rolling updates** apply when a **configuration change replaces instances**: instance type, AMI or platform version, VPC or subnet settings, launch template settings such as the key pair or root volume.
- Namespace `aws:autoscaling:updatepolicy:rollingupdate`:
  - `RollingUpdateEnabled`.

  - `RollingUpdateType`: **Time** (waits `PauseTime` between batches), **Health** (waits for health checks), or **Immutable** (temporary ASG).
  - `MaxBatchSize`, `MinInstancesInService`, `PauseTime`.

Health

	Basic health	Enhanced health
Source	ELB health checks and EC2 status	A **health agent** on each instance also reports requests, 4xx/5xx, latency, CPU, load
Colors	Green, Yellow, Red, Grey	Green OK, Yellow Warning, Red Degraded or Severe, Grey Unknown
Needs	Nothing	Service role and instance profile. **Required for managed platform updates and immutable updates**
CloudWatch	Basic	Can publish health metrics as **custom metrics** (extra cost)

- Set the **health check URL** (for example `/health`) so the ELB checks the app, not just the port.
- Deployments use health to decide whether a batch, or the immutable instance, succeeded.

Managed platform updates

- Beanstalk applies **minor and patch** platform updates automatically in a **weekly maintenance window**.
- It uses an **immutable** deployment, so it **needs enhanced health**.
- **Instance replacement** is an option that replaces instances on a schedule even when there is no update.
- **Major** platform branch changes (for example AL2 to AL2023) are **not** done in place. Use **blue/green** with swap URLs.

Part 3 - .ebextensions (configuring the environment from code)

- A folder named `.ebextensions/` at the **root** of the source bundle, holding files ending in `.config`, written in **YAML or JSON**.
- Files are processed in **alphabetical order**, for example `01-packages.config` then `02-options.config`.
- They travel with the code, so configuration is version-controlled.

Keys in a .config file

Key	Purpose
`option_settings`	Set any Beanstalk option (namespace, option name, value)
`packages`	Install packages: yum, rubygems, python (pip) and others
`groups` / `users`	Create Linux groups and users
`sources`	Download an archive and extract it into a folder
`files`	Create files on the instance with given content, owner and mode
`commands`	Shell commands **before** the app is extracted, in alphabetical order by name
`services`	Start, stop or enable services
`container_commands`	Commands **after** the app and web server are set up but **before** the new version goes live
`Resources`	Extra **CloudFormation resources** (DynamoDB table, SQS queue, SNS topic, alarms)
`Outputs`	CloudFormation outputs

- **Execution order:** packages, groups, users, sources, files, commands, services, container_commands.

container_commands options

- `leader_only: true`: runs on **only one instance**. The exam answer for **database migrations** or one-time setup.
- `test`: runs only if the test command succeeds.
- `ignoreErrors`: carries on even if the command fails.

```
# .ebextensions/01-setup.config
option_settings:
  aws:elasticbeanstalk:application:environment:
    APP_ENV: production
  aws:elasticbeanstalk:command:
    DeploymentPolicy: Immutable
  aws:autoscaling:launchconfiguration:
    InstanceType: t3.medium

packages:
  yum:
    jq: []
files:
  "/etc/myapp/settings.json":
    mode: "000644"
    owner: root
    group: root
    content: |
      {"log_level": "info"}

commands:
  01_mkdir:
    command: mkdir -p /var/myapp/cache
```

```
container_commands:
  01_migrate:
    command: "python manage.py migrate"
    leader_only: true

Resources:
  OrdersTable:
    Type: AWS::DynamoDB::Table
    Properties:
      BillingMode: PAY_PER_REQUEST
      AttributeDefinitions: [{AttributeName: id, AttributeType: S}]
      KeySchema: [{AttributeName: id, KeyType: HASH}]
```

Settings priority (highest wins)

1. Settings applied **directly to the environment** (console, EB CLI or API: `eb setenv`, `eb config`, `UpdateEnvironment`).
2. **Saved configurations**.
3. **.ebextensions** config files.
4. Defaults.

- Exam trap: you changed the instance type in `.ebextensions` but nothing changed. A value set earlier in the console or CLI **overrides** it. Remove the direct setting.

Resources tied to the environment lifecycle

- Anything created in `Resources`, or an RDS database created **inside** the environment, is **deleted when the environment is terminated**.
- For production, **create RDS outside Beanstalk** and pass the endpoint through environment properties or Secrets Manager.
- To move a database out:
  1. Take a snapshot and turn on deletion protection.
  2. Create a new environment that points to a database restored from the snapshot.
  3. Swap to the new environment.
  4. Terminate the old one, keeping the RDS database (Retain).

Other config files in the bundle

File or folder	Purpose
`.platform/hooks/prebuild/`, `predeploy/`, `postdeploy/`	Scripts that run at those stages (Amazon Linux 2 and later)
`.platform/confighooks/`	The same hooks, for **configuration-only** deployments
`.platform/nginx/conf.d/*.conf`	Extend the nginx proxy configuration (upload size, timeouts)
`Procfile`	Defines the processes that start your app, for example `web: gunicorn app:app`
`Buildfile`	Build commands that run on the instance
`env.yaml`	**Environment manifest**: name, platform, tier, CNAME prefix, options and **environment links**
`cron.yaml`	**Periodic tasks** for worker environments

Common configuration tasks

- **HTTPS:** `option_settings` on `aws:elbv2:listener:443` with an ACM certificate ARN. Redirect HTTP to HTTPS with an ALB listener rule or nginx configuration.
- **Logs to CloudWatch:** `aws:elasticbeanstalk:cloudwatch:logs` with `StreamLogs: true`, `RetentionInDays` and `DeleteOnTerminate`. Health logs use `aws:elasticbeanstalk:cloudwatch:logs:health`. Logs can also rotate to S3 every hour.

- **X-Ray:** `aws:elasticbeanstalk:xray` with `XRayEnabled: true` runs the X-Ray daemon. The instance profile needs X-Ray write permissions.
- **Secrets:** never in `.ebextensions` or plain environment properties in the repository. Beanstalk can reference **Secrets Manager secrets and SSM Parameter Store parameters as environment variables**, or the app reads them at runtime using the instance profile.
- **Custom AMI:** `aws:autoscaling:launchconfiguration ImageId` for faster startups, with software baked in using EC2 Image Builder.

Part 4 - Swap environment URLs (blue/green on Beanstalk)

Blue/green is **not** a deployment policy. It is a process using **two environments**:

1. **Blue** is the current production environment with URL `myapp.us-east-1.elasticbeanstalk.com`.
2. Create **green**: clone blue (`eb clone`) or create a new environment, and deploy the new version or platform to it.
3. Test green on its own URL.
4. **Swap environment URLs** (`eb swap`, or the API `SwapEnvironmentCNAMEs`). Beanstalk **swaps the CNAME records**.
5. Keep blue running until DNS caches expire. To **roll back**, swap again.
6. When you are confident, terminate blue.

Key facts

- It is a **DNS change**, so it is not instant for everyone. Clients and resolvers can cache the old record (TTL), so **don't terminate blue right away**.
- Both environments must be in the **same application** and the **same Region**, and both must be running.
- With Route 53 and your own domain, point the record at the **Beanstalk environment URL** (not the ELB DNS name), so the swap takes effect.
- **Database:** must be **outside** both environments so blue and green share the same data.
- **Gradual shift:** use **Route 53 weighted records** pointing at the two environment URLs (90/10, 50/50, 0/100).

When the answer is blue/green with swap URLs

- Changing **platform branch or major version** (for example AL2 to AL2023).
- Changing the **load balancer type** (Classic to ALB), or other settings that can't change in place.
- Needing a **completely separate** environment to test before going live, with the **fastest rollback** (swap back).

	Swap URLs	Immutable	Traffic splitting
Separate environment	**Yes**	No (temporary ASG)	No (temporary ASG)
Switch mechanism	DNS CNAME	ASG move	ALB weighted
Gradual	No (unless Route 53 weights)	No	**Yes**, a percentage for a set time
Platform branch or LB type change	**Yes**	No	No
Rollback	Swap back (DNS delay)	Terminate the temporary ASG	Automatic

Part 5 - Worker environments

Purpose: move **long-running or background work** out of the web tier: video encoding, reports, sending email, image processing.

How it works

1. The **web tier** (or anything else) sends a message to an **SQS queue**.
2. Each worker instance runs the **SQS daemon (`sqsd`)**, which **polls the queue**.
3. For each message, `sqsd` sends an **HTTP POST** with the message body to your app at `http://localhost/` (path configurable).
4. If your app returns **HTTP 200 OK**, `sqsd` **deletes the message**.
5. Any other response or a timeout: the message **becomes visible again** after the visibility timeout and is retried.
6. After the **maximum retries**, the message goes to the **dead-letter queue (DLQ)**.

- Worker environments have **no load balancer** and no public URL.
- The queue can be **created automatically** (with a DLQ), or you give **your own queue URL**.
- **FIFO queues are supported**, but periodic tasks (cron) don't work with FIFO.

sqsd settings (namespace aws:elasticbeanstalk:sqsd)

Option	Meaning
`WorkerQueueURL`	The queue to read (blank means auto-create)
`HttpPath`	The path to POST to (default `/`)
`MimeType`	The content type of the POST
`HttpConnections`	Maximum concurrent connections to your app per instance
`ConnectTimeout` / `InactivityTimeout`	How long to wait for your app. **Raise `InactivityTimeout` for long jobs**
`VisibilityTimeout`	How long a message is hidden while processing. Must be **longer than the job**, or it runs twice
`ErrorVisibilityTimeout`	The delay before retrying after an error
`RetentionPeriod`	How long a message is valid. Older messages are discarded
`MaxRetries`	Attempts before sending to the DLQ (auto-created queue)

Periodic tasks (cron.yaml)

```
version: 1
cron:
  - name: "nightly-report"
    url: "/tasks/report"
    schedule: "0 2 * * *"      # UTC
  - name: "cleanup"
    url: "/tasks/cleanup"
    schedule: "*/30 * * * *"
```

- Put `cron.yaml` at the root of the **worker** environment's source bundle.
- Beanstalk uses **leader election** with a **DynamoDB table**. Only **one instance** adds the scheduled message to the queue, so each task runs **once**, not once per instance.
- The instance profile needs DynamoDB permissions for the leader election table.

Scaling and connections

- By default the worker ASG scales on CPU. Better: scale on **queue depth**, with a CloudWatch alarm on `ApproximateNumberOfMessagesVisible` (added through `.ebextensions` `Resources` or

`aws:autoscaling:trigger`).

- **Environment links** (in `env.yaml`) connect web and worker environments, so the web tier gets the queue URL as an environment variable.
- The web tier's instance profile needs `sqs:SendMessage`. The worker's instance profile needs receive and delete permissions (the managed worker policy).

The question says	Answer
Requests time out because processing takes minutes	Send the work to a **worker environment** through SQS
Scheduled job runs on every instance, giving duplicates	`cron.yaml` in a worker environment (leader election)
Job processed twice	Raise **VisibilityTimeout** above the job duration
Failed messages retried forever	Use a **DLQ** or `MaxRetries`
Messages piling up	**Scale on queue depth**

Part 6 - CI/CD, monitoring and troubleshooting

CI/CD

- **CodePipeline** has an **Elastic Beanstalk deploy action**. It deploys the build artifact as a new application version, using the environment's deployment policy.
- With CodeBuild, the output artifact must be a valid source bundle, with `.ebextensions` at its root.
- Beanstalk can be defined in **CloudFormation**: `AWS::ElasticBeanstalk::Application`, `ApplicationVersion`, `ConfigurationTemplate` and `Environment`.

Docker platforms

- **Docker on AL2 / AL2023:** a `Dockerfile` or `docker-compose.yml`. It can pull images from ECR (the instance profile needs ECR permissions).
- **ECS on AL2 (multi-container):** `Dockerrun.aws.json` version 2, running an ECS cluster under the hood.

Monitoring and logs

- Events, enhanced health and CloudWatch alarms.
- Beanstalk **notifications through SNS**, and **EventBridge** events for state changes.
- **Request logs:** last 100 lines (tail) or a **full bundle** (`eb logs`). Logs can **stream to CloudWatch Logs** or **rotate to S3**.

Troubleshooting

Symptom	Cause and fix
Deployment fails with a command timeout	Raise `Timeout` in `aws:elasticbeanstalk:command` (default 600 seconds)
New instances fail health checks and roll back	Wrong health check URL, app not listening on the expected port, or missing instance profile permissions
Changes in `.ebextensions` are ignored	A direct environment setting overrides it, or the file isn't in `.ebextensions/` at the bundle root, doesn't end in `.config`, or has invalid YAML
Deployment fails because of the version limit	Set an **application version lifecycle policy**
Database gone after environment terminated	RDS was inside the environment. Keep RDS outside Beanstalk
Someone changed the ASG or ELB in the EC2 console and it broke	Drift from the CloudFormation stack. Change it through Beanstalk options
Migration ran on every instance	`container_commands` with `leader_only: true`

Quick memory summary

- **5 policies:** All at once (fast, downtime), Rolling (reduced capacity), Rolling with additional batch (full capacity), Immutable (temporary ASG, safest rollback), Traffic splitting (ALB canary, automatic rollback).
- **Configuration changes** use **rolling updates** (Time, Health or Immutable), not deployment policies.
- **.ebextensions:** `.config` files in YAML or JSON at the bundle root, processed alphabetically. `leader_only` runs a command once. Direct settings override `.ebextensions`. Resources are deleted with the environment.
- **Swap URLs:** a CNAME (DNS) swap between two environments in the same application and Region. For platform branch or LB type changes, and instant rollback by swapping back. Keep the database outside.
- **Worker:** SQS, `sqsd`, HTTP POST to localhost. 200 deletes the message; anything else retries, then the DLQ. `cron.yaml` with leader election runs periodic tasks once. Keep the visibility timeout longer than the job, and scale on queue depth.